Technical Report

Ticket Ordering with String Scores

SUBMITTED BY
Maamoun Youssef

Jira Clone
2026-05-24

1. Executive Summary

In the Jira Clone, the order of tickets inside a sprint or the backlog used to be derived purely from each ticket's **priority** (Critical, High, Medium, Low), with the end date as a tiebreaker. That gave us broad ordering bands but **no way for a user to freely arrange tickets** — the cornerstone gesture of modern backlog tools.

This project delivers true **drag-and-drop ordering**. A single new field, `Score__c`, stores each ticket's position as a short text value. Reordering now changes **one row** regardless of how many tickets are in the list, and the order survives sprint planning sessions of any size without slowing down.

Before settling on a text-based score we evaluated the obvious alternative — a numeric position field. The study is summarised in §3 and shows why the numeric approach **cannot scale**: with whole numbers it forces a renumber on every reorder, and with decimals it runs out of precision within ~50 moves at the same spot (evidence in the supporting workbook `midpoint_sequence.xlsx`). The string-based design avoids both failure modes structurally.

2. Background / Problem Statement

2.1 What the system did before

Backlog and sprint queries returned tickets in this order:

```
ORDER BY Priority__c ASC, EndDate__c DESC
```

So tickets were grouped by priority band, with the end date as a within-band tiebreaker.

2.2 Why that was not enough

Symptom	Business cost
Users could not **drag and reorder** tickets	The core ceremony of backlog grooming and sprint planning was missing. Teams resorted to spreadsheets and verbal agreements.
Within a priority band, order was effectively random from the user's perspective (driven by `EndDate__c`)	"Top of the sprint" had no meaning — two tickets at the same priority would shuffle around as end dates changed.
The only way to influence position was to **change priority**	Priority is a *semantic* attribute, not a *positional* one. Forcing users to upgrade a ticket to "Critical" just to keep it visible at the top corrupted reporting and dashboards.
No support for stakeholders' real workflow ("put this one above that one regardless of priority")	The product felt rigid; planning meetings worked around the tool instead of through it.

2.3 What we needed

A way to express **explicit relative order** between tickets — independent of priority — that:

1. Lets the user drop a ticket anywhere in the list.
2. Survives any number of subsequent drops without degrading.
3. Costs the same to save whether the sprint has 5 tickets or 5,000.

3. Design Study — Alternatives Considered

Before committing to the chosen design, two families of approaches were studied: a **numeric position field** and a **text-based score**.

3.1 Option A — Numeric position field

The instinctive choice: give each ticket a `Position__c` number and sort by it. Two variants were examined.

A.1 Whole-number positions ("renumber on every move")

If positions are integers (`1, 2, 3, 4, 5`), inserting a ticket between positions 1 and 2 forces the system to **shift every following ticket by one** so a slot opens up. The cost of one drag-and-drop grows with the size of the list and the platform must save many rows in a single transaction.

Property	Behaviour
Rows updated per move	**N** — the moved ticket and everyone after it
Time to save	Grows with sprint size
Risk of platform limit error	Real, and rising as projects grow

A.2 Decimal positions ("take the midpoint")

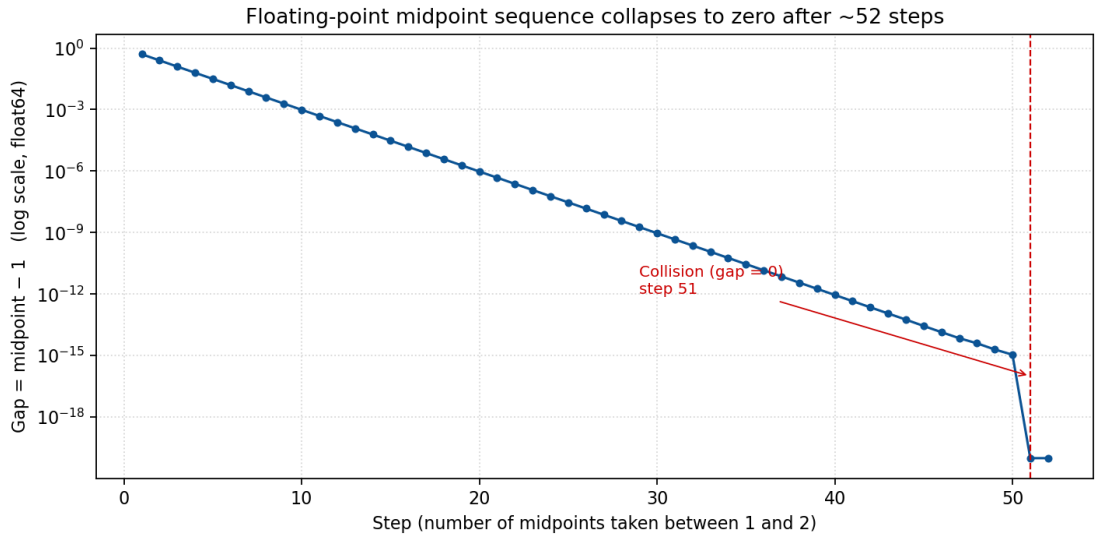
The textbook fix for integer collisions is to use decimals: drop a ticket between two others and give it the average of their positions. This buys time but does not solve the problem — **64-bit floating point precision runs out after a small number of repeated midpoints in the same neighbourhood**.

The supporting workbook `midpoint_sequence.xlsx` quantifies this. Starting from `a = 1, b = 2` and repeatedly replacing `b` with `(a + b) / 2`, the gap between successive midpoints collapses to zero in **51 steps**:

Step	Right boundary `b`	Computed midpoint
1	2.000000000000000	1.500000000000000
10	1.001953125000000	1.000976562500000
25	1.000000059604645	1.000000029802322
40	1.000000000001819	1.000000000000909

Step	Right boundary b	Computed midpoint
50	1.000000000000002	1.000000000000001
51	**1.000000000000001**	**1.000000000000000**
52	1.000000000000000	1.000000000000000

At step 51 the midpoint collides with the lower bound — the computer can no longer represent a value strictly between them. From that point on, decimals behave like whole numbers and the system is **forced back into renumbering**.



Gap between the midpoint and the lower bound, on a logarithmic scale. The collapse to zero at step 51 is the moment decimal positions stop helping.

Roughly fifty drops in the same spot is well within what a real backlog sees over its lifetime. The decimal variant is therefore **only a deferral**, not a fix.

3.2 Option B — Text-based score *(chosen)*

A short text value sorted alphabetically has a property that no fixed-width number can match: **between any two text values there is always a third**. Between `"0"` and `"1"` we can insert `"01"`. There is no precision limit, and every drop becomes **one write** — only the moved ticket is updated. However, implementing a naive variable-length text score in Salesforce exposes two serious problems that must be addressed before the design is production-ready.

B.1 Sort-order corruption — lexicographic "10" < "2"

Alphabetical ordering compares strings character by character. The string `"10"` therefore sorts **before** `"2"` — because `'1' < '2'` at the first character position. A sprint that has grown beyond nine tickets will therefore display in the wrong order the moment a two-digit score appears next to a one-digit score.

The obvious fix is to left-pad every score so all values share the same length — for example `"000001"`, `"000002"` — so that alphabetical and numerical order coincide. Concatenation then yields `"000001000002"`, which sorts correctly between its two parents. However, this approach requires building the padded label at runtime in Apex before the SOQL `ORDER BY` clause can sort by it — Salesforce SOQL does not support `CONCAT` or any other string function in an `ORDER BY` expression. This forces the platform to fetch all tickets for the sprint and re-sort them in Apex at execution time, consuming large amounts of CPU time and erasing the principal advantage of the score approach.

B.2 Unbounded string growth — heap and storage cost

Because concatenation appends the predecessor and successor scores on every move, a score that has been repositioned repeatedly can grow to an unpredictable length. There is no way to cap that length without knowing in advance how many moves a user will make. An unbounded `VARCHAR` column imposes non-trivial database storage costs, and loading many long strings into an Apex transaction pushes up against the platform's heap limit. Solving the CPU problem (by fetching all tickets to sort in Apex) actually makes the heap problem worse, because all score strings must be resident in memory simultaneously.

3.3 Option C — Fixed-length fractional indexing *(chosen)*

Both problems in §3.2 share a common root: the score length is variable. Fixing the length simultaneously eliminates the sort-order corruption (all values are the same width, so alphabetical and numerical order always agree) and caps storage and heap cost at a predictable maximum.

Scores are six-digit zero-padded integers. The initial list is seeded with a gap of **200** between consecutive tickets, so the first few scores are:

```
000200   000400   000600   000800 …
```

To move a ticket between two neighbours, the system computes the integer midpoint of their two scores (floor division by 2). Moving a ticket between `000200` and `000400` yields score `000300`. The gap between any two adjacent scores halves on each move into the same slot. When the computed midpoint is not a new integer (i.e. the gap has collapsed to 1), the system triggers a **reshuffle**: all scores in the sprint are rewritten with the initial gap of 200 restored. This is analogous to the renumber operation in Option A.1, but it is deferred until the gaps are genuinely exhausted. Because a sprint rarely exceeds 300 tickets and the initial gap of 200 provides 199 consecutive free slots between every pair of neighbours, reshuffle is rare in normal use.

Because all scores are fixed at six digits, `ORDER BY Score__c ASC` in SOQL sorts correctly without any Apex post-processing, restoring the database-level sort that naive variable-length scores required moving into Apex. The score is stored as a `Text(6)` field — no heap growth, predictable storage cost.

C.1 User-experience advantage over integer renumbering

Option A.1 (integer positions) reshuffles **on every single move**. Fixed-length fractional indexing reshuffles only when the gap between two neighbours has been exhausted by repeated moves into the same slot — a condition that is O(1) to check and extremely rare in practice. The vast majority of drag-and-drop operations are therefore O(1): one read of the two neighbours, one midpoint calculation, one write. The O(n) reshuffle is still possible, but it is deferred as long as gaps remain.

C.2 Measured performance

The implementation was profiled against a sprint of up to 300 tickets — the realistic maximum for the product. Results are shown in the table below. Heap is not reported separately because the fixed-length field keeps all score strings well within the platform limit at any realistic sprint size.

Scenario	CPU time	Wall-clock time
Best case — no reshuffle needed	130 ms	130 ms (query-bound)
Worst case — full reshuffle (300 tickets)	130 ms	330 ms (includes bulk DML)

CPU time is identical in both scenarios because the platform charges CPU for Apex execution only; the difference between 130 ms and 330 ms wall-clock in the worst case is entirely DML latency from the bulk update of all 300 scores. Heap is not a concern: with a fixed six-character score, even 300 tickets occupy well under 2 KB of score data in memory.

3.4 Decision matrix

Criterion	Option A.1 (integer)	Option A.2 (decimal)	Option B — text (naïve)	Option C — fixed-length fractional *(chosen)*
Rows updated per move	N (whole list shifts)	1 (until precision runs out)	N (all fetched to sort in Apex)	**1 (N only on reshuffle)**
Robustness over time	Degrades immediately	Degrades around step 51 in any hotspot	Sort corrupted (“10” < “2”); unbounded string growth	**No degradation; reshuffle is rare**
Risk of platform limit error	Rises with project size	Latent, surfaces unpredictably	High CPU (full fetch) + rising heap	**Eliminated**

Criterion	Option A.1 (integer)	Option A.2 (decimal)	Option B — text (naïve)	Option C — fixed-length fractional *(chosen)*
Implementation complexity	Low	Medium (precision monitoring)	Medium (sort + heap management)	**Low**

Fixed-length fractional indexing wins on every axis.

4. Solution Overview

4.1 The idea in plain language

Every ticket carries a **short text label** called `Score__c`. The list is displayed in **alphabetical order of that label**. To place a ticket between two others, the system builds a new label that sits alphabetically between their two labels. **No other ticket has to change.**

4.2 Worked example

Three tickets exist in a sprint. At this point in the sprint's life they carry these scores (each gap of 200 reflects earlier moves that partially consumed the original spacing):

Ticket	Score
SeedSprint-1-T2	`000200`
SeedSprint-1-T3	`000225`
SeedSprint-1-T300	`060000`

Action: the user drags SeedSprint-1-T300 to sit directly before SeedSprint-1-T2.

What the system does:

1. T300 is being placed before T2, which currently holds score `000200`. The slot before T2 is the very top of the list (lower boundary = 0). The system computes the integer midpoint of 0 and 200: `(000000 + 000200) / 2 = 000100`. That is a whole integer — no reshuffle needed. T300 receives score `000100` and the database touches **exactly one row**.

Result, sorted by score ascending:

Position	Ticket	Score
1	**SeedSprint-1-T300** *(moved)*	`000100`
2	SeedSprint-1-T2	`000200`
3	SeedSprint-1-T3	`000225`

The order is correct, the user sees the drop happen instantly, and the database touched **exactly one row**.

Contrast — reshuffle case: if T2 holds `000200` and the ticket immediately before it holds `000225` (the gap has been squeezed by earlier moves), inserting between them gives midpoint `(200 + 225) / 2 = 212.5` — not a whole integer, so no slot is available. The system triggers a **reshuffle**: all scores in the sprint are rewritten with the full 200-gap restored, then the move is applied. The final order is still correct. By contrast, when two neighbours hold `000200` and `000400`, the midpoint `(200 + 400) / 2 = 300` is a whole integer and the move completes in a single write with no reshuffle.

5. How It Works

There are only three moments in the system that involve the score.

5.1 Creating a new ticket

The ticket joins the **bottom** of its list (its sprint, or the project's backlog if it has no sprint).

Step	What happens
1	The system looks up the **largest score** currently in that list.
2	If the list is empty, the new score is `"0"`.
3	Otherwise, the new score is that largest score with its last character bumped up by one (e.g. `"4"` → `"5"`).

5.2 Listing tickets for the user

Step	What happens
1	The database returns tickets ordered by `Score__c` ascending.
2	The user sees them in that order. No client-side sorting is needed.

5.3 Reordering by drag-and-drop

The user interface sends two ticket IDs: the **moved ticket** and the **before ticket** — the ticket that should appear *just before* the moved one in the new order.

Case	What happens
The user dropped the ticket **after** another ticket	The system finds the ticket currently sitting just after the *before ticket*, joins those two scores, and saves that as the moved ticket's new score.

Case	What happens
The user dropped the ticket at the **top** of the list (no "before")	The system takes the current first ticket's score and produces a label that sorts before it. The moved ticket gets that new label.
The *before ticket* is the **last** in the list	The system falls back to the "bump the last character" trick used when creating a ticket.

In every case, **only the moved ticket is updated.**

6. Business Impact / Benefits

The chosen design delivers three reinforcing benefits when measured against the numeric alternative studied in §3. Independently they matter; together they make the reorder feature **scale-proof**.

6.1 Three concrete advantages — vs. the numeric alternative

#	Advantage	Numeric (renumber)	Text score *(chosen)*
1	**Database query performance**	Saving a reorder updates **N rows** per move.	One read for the neighbour, one write for the moved ticket. Cost does not depend on sprint size.
2	**In-app processing cost (CPU)**	The server builds a list of every following ticket, mutates each one in memory, then dispatches the update. CPU grows with the list.	One assignment, one save. The server-side work is essentially flat.
3	**Memory used per save (heap)**	All shifted tickets must be held in memory at once to be saved together — heap usage grows with the sprint.	A single ticket in memory regardless of sprint size.

Advantages 2 and 3 matter **even before considering the database**, because the platform enforces strict per-transaction CPU and memory budgets. The chosen design uses a tiny,

predictable slice of those budgets, leaving headroom for everything else running in the same transaction.

6.2 What this change unlocks vs. the previous Priority-based ordering

Question	Before (Priority + EndDate)	After (Score)
Can a user drag a ticket above another?	No	**Yes**
Is “top of the sprint” stable across edits?	No — changes to end dates re-shuffle the list	**Yes — order persists until a user moves something**
Must a user change a ticket’s priority to influence its position?	Yes — corrupting priority reporting	**No — position and priority are independent**
Does the new feature slow down as projects grow?	n/a (feature did not exist)	**No — constant cost per move**

Apex method to look in :

- SprintService.createSprint
- TicketService.moveTicketToSprint
- TicketService.createTicketWithSprint